StudyGenie AI — Data Structures & Algorithms Notes

Compiled from the four chapter notes provided for this project.

Chapter 1: Introduction, Time & Space Complexity, Big-O

1.1 What Are Data Structures and Algorithms?

A **data structure** is a way of organizing and storing data in memory (array, linked list, tree, graph, and so on).

An **algorithm** is a step-by-step procedure for solving a problem (sorting, searching, dynamic programming, and so on).

The two work together. The right data structure combined with the right algorithm gives you a program that is both fast and memory-efficient.

Real-life example: Searching for a name in your phone's contact list. If the contacts were in random order, you would have to check them one by one (slow). Because they are sorted alphabetically, you can jump to the middle and narrow down quickly (fast). That idea is the heart of DSA.

Why learn DSA?

• It is the most common topic in technical interviews.

• It makes your code faster and more scalable.

• It sharpens your problem-solving skills.

• Every large system, from maps to social feeds to search engines, relies on it.

1.2 Properties of a Good Algorithm

1. **Correctness:** gives the right output for every valid input.

2. **Finiteness:** terminates after a limited number of steps.

3. **Efficiency:** uses little time and memory.

4. **Clarity:** is easy to understand and maintain.

1.3 Java Refresher for DSA

Basic program structure

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello DSA");
    }
}
```

Data types you will use constantly

Type	Size	Range / Use
`int`	32 bit	-2^31 to 2^31-1 (about ±2.1 x 10^9)
`long`	64 bit	about ±9.2 x 10^18, for large sums
`double`	64 bit	decimal values
`char`	16 bit	a single character
`boolean`	logical	true / false

Important: If an answer can exceed 10^9, use `long`. Otherwise you will hit **integer overflow**.

```
int a = Integer.MAX_VALUE;
System.out.println(a + 1);        // -2147483648 (overflow!)
long b = (long) a + 1;
System.out.println(b);            // 2147483648 (correct)
```

Quick look at arrays and collections

```
int[] arr = new int[5];                // size 5, defaults to 0
int[] arr2 = {1, 2, 3, 4, 5};

import java.util.*;
ArrayList<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
System.out.println(list.get(0));       // 10
System.out.println(list.size());       // 2
```

Reading input

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }
        System.out.println(Arrays.toString(a));
    }
}
```

For large inputs (more than about 10^5 numbers), use `BufferedReader`. `Scanner` is slow.

```
import java.io.*;
import java.util.*;

public class FastInput {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());
        StringTokenizer st = new StringTokenizer(br.readLine());
        int[] a = new int[n];
        for (int i = 0; i < n; i++) {
            a[i] = Integer.parseInt(st.nextToken());
        }
        System.out.println(Arrays.toString(a));
    }
}
```

1.4 What Is Complexity Analysis?

A single problem can usually be solved in several ways. How do we decide which is better? Not by measuring seconds, since that depends on the machine. Instead we measure **how the work grows as the input size (n) grows**.

• **Time complexity:** how the number of operations grows with n.

• **Space complexity:** how the extra memory used grows with n.

Example: sum of 1 to n

Approach 1: loop

```
static long sumLoop(int n) {
    long sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    return sum;
}
```

Time: **O(n)**, Space: **O(1)**

Approach 2: formula

```
static long sumFormula(int n) {
    return (long) n * (n + 1) / 2;
}
```

Time: **O(1)**, Space: **O(1)**

For n = 10^9, the first approach takes about a billion steps, while the second takes one. That is the difference complexity makes.

1.5 Asymptotic Notations

Big-O (O): upper bound, usually the worst case

"The algorithm will **never be slower** than this (up to a constant factor)."

Formally, f(n) = O(g(n)) if there exist constants c > 0 and n0 such that f(n) <= c * g(n) for all n >= n0.

Big-Omega (Ω): lower bound, usually the best case

"The algorithm needs **at least** this much time."

Big-Theta (Θ): tight bound

Used when the upper and lower bounds match.

In interviews, Big-O (worst case) is what you will be asked about most.

Simplification rules

1. **Drop constants:** O(3n) becomes O(n).

2. **Drop lower-order terms:** O(n^2 + n) becomes O(n^2).

3. **Use different variables for different inputs:** looping over array a (size n) and array b (size m) is O(n + m), not O(n).

4. **Nested loops multiply, sequential loops add.**

1.6 Common Time Complexities (smallest to largest)

Complexity	Name	Example
O(1)	Constant	Accessing an array element by index
O(log n)	Logarithmic	Binary search
O(n)	Linear	Finding the max of an array
O(n log n)	Linearithmic	Merge sort, heap sort
O(n²)	Quadratic	Bubble sort, double nested loops
O(n³)	Cubic	Three nested loops, naive matrix multiplication
O(2^n)	Exponential	Generating all subsets
O(n!)	Factorial	Generating all permutations

What complexity fits which input size? (rule of thumb)

Assume roughly 10^8 simple operations per second.

Constraint on n	Complexity that will pass
n <= 10	O(n!)
n <= 20	O(2^n)
n <= 500	O(n³)
n <= 5000	O(n²)
n <= 10^6	O(n log n) or O(n)
n <= 10^9	O(log n) or O(1)

This table is extremely handy in interviews and contests: read the constraints and you can often guess the intended approach.

1.7 Finding Complexity from Code

Example 1: single loop

```
for (int i = 0; i < n; i++) {
    System.out.println(i);
}
```

O(n)

Example 2: nested loop

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.println(i + j);
    }
}
```

The inner loop runs n times for each of the n outer iterations: n x n = **O(n²)**.

Example 3: sequential loops

```
for (int i = 0; i < n; i++) { /* work */ }
for (int j = 0; j < n; j++) { /* work */ }
```

n + n = 2n = **O(n)**

Example 4: dependent nested loop

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.println(i + j);
    }
}
```

Inner iterations: 0 + 1 + 2 + ... + (n-1) = n(n-1)/2, which is **O(n²)**.

Example 5: loop that multiplies

```
for (int i = 1; i < n; i *= 2) {
    System.out.println(i);
}
```

i takes the values 1, 2, 4, 8, ... up to n. That is k steps where 2^k is about n, so k = log■(n). Complexity: **O(log n)**.

Example 6: square-root loop

```
for (int i = 1; i * i <= n; i++) {
    System.out.println(i);
}
```

The loop runs while i² <= n, meaning i <= √n. Complexity: **O(√n)**.

Example 7: nested loop with a log inside

```
for (int i = 0; i < n; i++) {
    for (int j = 1; j < n; j *= 2) {
        System.out.println(i + j);
    }
}
```

n x log n = **O(n log n)**

Example 8: two different inputs

```
for (int i = 0; i < a.length; i++) {
    for (int j = 0; j < b.length; j++) {
        // work
    }
}
```

O(n x m). Do not write O(n²) here.

1.8 Space Complexity

For space, we count the **extra memory** an algorithm uses (this is called auxiliary space). The input itself is usually not counted.

```
// O(1) space
static int findMax(int[] a) {
    int max = a[0];
    for (int x : a) max = Math.max(max, x);
    return max;
}

// O(n) space
static int[] copyArray(int[] a) {
    int[] b = new int[a.length];
    for (int i = 0; i < a.length; i++) b[i] = a[i];
    return b;
}

// O(n) space (recursion stack)
static int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

Remember: every recursive call occupies space on the call stack. In `factorial(n)`, n calls are on the stack at the same time, so the space is **O(n)** even though no array is created.

1.9 Complexity of Recursion (Intro)

Linear recursion

```
static int sum(int n) {
    if (n == 0) return 0;
    return n + sum(n - 1);
}
```

Recurrence: T(n) = T(n-1) + O(1), so **O(n)** time and **O(n)** space.

Binary recursion (naive Fibonacci)

```
static int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Each call makes two more calls, and the tree is n levels deep. Time is **O(2^n)** (more precisely about O(1.618^n)). Space is **O(n)**.

Halving the problem (binary-search style)

T(n) = T(n/2) + O(1) gives **O(log n)**.

Merge-sort style

T(n) = 2T(n/2) + O(n) gives **O(n log n)**.

Master Theorem (quick reference)

For T(n) = a·T(n/b) + O(n^d):

• If d > log_b(a): **O(n^d)**

• If d = log_b(a): **O(n^d · log n)**

• If d < log_b(a): **O(n^(log_b a))**

Example: merge sort has a = 2, b = 2, d = 1. Since log_2(2) = 1 = d, the result is **O(n log n)**.

1.10 Best, Average and Worst Case

Linear search example:

```
static int linearSearch(int[] a, int key) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == key) return i;
    }
    return -1;
}
```

• **Best case:** the key is at index 0, so O(1).

• **Worst case:** the key is last or missing, so O(n).

• **Average case:** about n/2 comparisons, so O(n).

1.11 Amortized Analysis (Intro)

Sometimes a single operation is occasionally expensive, but **averaged over many operations** it is cheap.

Example: `ArrayList.add()`

When the internal array is full, Java allocates a larger array (roughly 1.5x) and copies every element. That one call is O(n). But most `add()` calls are O(1), and n adds cost O(n) in total, so each add is **amortized O(1)**.

1.12 Common Mistakes

1. **Ignoring huge constants.** An O(n) algorithm with a heavy per-step cost can lose to an O(n log n) one on small inputs.

2. **Forgetting recursion stack space.**

3. **String concatenation in a loop.** Java strings are immutable, so `s += x` creates a new string each time. Doing it n times costs **O(n^2)**. Use `StringBuilder`.

```
// Slow: O(n^2)
String s = "";
for (int i = 0; i < n; i++) s += "a";

// Fast: O(n)
StringBuilder sb = new StringBuilder();
for (int i = 0; i < n; i++) sb.append("a");
String result = sb.toString();
```

4. **Treating `list.remove(0)` or `list.contains()` as O(1).** Both are O(n) on an ArrayList.

5. **Ignoring integer overflow.**

1.13 Chapter Summary

• DSA means organizing data and defining the steps to solve problems on it.

• Complexity measures how time and space grow with input size.

• Big-O gives the worst-case upper bound.

• Drop constants and lower-order terms.

• Nested loops multiply; sequential loops add.

• Use the input constraints to pick your approach.

• Never forget the recursion stack when counting space.

1.14 Practice Problems

Q1. Find the time complexity of this code.

```
int count = 0;
for (int i = n; i > 0; i /= 2) {
    for (int j = 0; j < i; j++) {
        count++;
    }
}
```

Q2. Find the time and space complexity of this code.

```
static int mystery(int n) {
    if (n <= 0) return 0;
    return mystery(n / 2) + mystery(n / 2);
}
```

Q3. What is the complexity of `for (i = 0; i < n; i++) for (j = i; j < n; j++) for (k = 0; k < 10; k++)`?

Q4. Are O(2n) and O(n) the same? Are O(n^2) and O(n^2 + n log n) the same?

Q5. Write a function that checks whether a number is prime in O(√n).

Q6. Write two ways to detect a duplicate in an array: one O(n^2) and one O(n) using a HashSet. State the space complexity of each.

Q7. What problems can arise when using `Math.pow(2, n)` in Java? (Hint: overflow and precision.)

1.15 Solutions

A1. The inner loop runs i times, and i takes the values n, n/2, n/4, ..., 1. The sum is about 2n, so the total is **O(n)**. (It is not O(n log n).)

A2. T(n) = 2T(n/2) + O(1). The recursion tree has about n leaves, so time is **O(n)**. The tree is log n deep, so stack space is **O(log n)**.

A3. The first two loops together run about n(n+1)/2 times, which is O(n^2). The third loop is a constant 10. Total: **O(n^2)**.

A4. Yes, $O(2n) = O(n)$. Yes, $O(n^2 + n \log n) = O(n^2)$ because n^2 dominates.

A5.

```
static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; (long) i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

Time **O(√n)**, space **O(1)**.

A6.

```
// O(n^2) time, O(1) space
static boolean hasDuplicateBrute(int[] a) {
    for (int i = 0; i < a.length; i++) {
        for (int j = i + 1; j < a.length; j++) {
            if (a[i] == a[j]) return true;
        }
    }
    return false;
}

// O(n) time, O(n) space
static boolean hasDuplicateSet(int[] a) {
    HashSet<Integer> seen = new HashSet<>();
    for (int x : a) {
        if (!seen.add(x)) return true;   // add() returns false if already present
    }
    return false;
}
```

This shows the **time-space tradeoff**: spending more memory to save time.

A7. `Math.pow` returns a `double`, which loses precision for large results. With integers, `1 << n` overflows for n >= 31 (`int`) or n >= 63 (`long`). For large powers use modular exponentiation or `BigInteger`.

Next: Chapter 2, Arrays and Strings

Chapter 2: Arrays and Strings

2.1 Arrays

An array stores a fixed number of elements of the same type in **contiguous memory**. Because of this layout, accessing any element by index is O(1).

```
int[] a = new int[5];            // [0, 0, 0, 0, 0]
int[] b = {4, 8, 15, 16, 23};
int n = b.length;                // length is a field, not a method
b[2] = 99;                       // O(1) write
```

Cost of common operations

Operation	Time
Access / update by index	O(1)
Search (unsorted)	O(n)
Search (sorted, binary search)	O(log n)
Insert / delete at end (dynamic array)	O(1) amortized
Insert / delete at index i	O(n) (elements must shift)

Useful `java.util.Arrays` methods

```
Arrays.toString(a);              // "[1, 2, 3]"
Arrays.sort(a);                  // O(n log n)
Arrays.fill(a, -1);              // fill with a value
int[] c = Arrays.copyOf(a, 10);  // copy, padded/truncated to length 10
int[] d = Arrays.copyOfRange(a, 1, 4);
Arrays.equals(a, c);             // compares contents (a == c compares references!)
Arrays.binarySearch(a, key);     // array must be sorted
```

Two-dimensional arrays

```
int[][] grid = new int[3][4];            // 3 rows, 4 columns
int[][] m = {{1, 2, 3}, {4, 5, 6}};

for (int i = 0; i < m.length; i++) {
    for (int j = 0; j < m[i].length; j++) {
        System.out.print(m[i][j] + " ");
    }
    System.out.println();
}
```

Printing a 2D array: `System.out.println(Arrays.deepToString(m));`

Dynamic array: `ArrayList`

```
ArrayList<Integer> list = new ArrayList<>();
list.add(5);              // O(1) amortized
list.add(0, 7);           // O(n), shifts everything
list.get(1);              // O(1)
list.set(1, 10);          // O(1)
list.remove(list.size() - 1);   // O(1)
list.remove(0);                 // O(n)
```

Note: `list.remove(int index)` and `list.remove(Object o)` are different. For `ArrayList<Integer>`, use `list.remove(Integer.valueOf(5))` to remove the value 5.

2.2 Basic Array Patterns

Reverse an array in place

```
        static void reverse(int[] a) {
            int l = 0, r = a.length - 1;
            while (l < r) {
                int t = a[l]; a[l] = a[r]; a[r] = t;
                l++; r--;
            }
        }
```

Time O(n), space O(1).

Rotate an array right by k (reversal trick)

Reverse the whole array, then reverse the first k elements, then reverse the rest.

```
        static void rotate(int[] a, int k) {
            int n = a.length;
            k %= n;
            reverse(a, 0, n - 1);
            reverse(a, 0, k - 1);
            reverse(a, k, n - 1);
        }
        static void reverse(int[] a, int l, int r) {
            while (l < r) { int t = a[l]; a[l++] = a[r]; a[r--] = t; }
        }
```

Time O(n), space O(1).

Find second largest

```
        static int secondLargest(int[] a) {
            int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
            for (int x : a) {
                if (x > first) { second = first; first = x; }
                else if (x > second && x != first) second = x;
            }
            return second;
        }
```

2.3 Prefix Sums

A prefix-sum array lets you answer **range-sum queries in O(1)** after O(n) preprocessing.

`prefix[i]` = sum of `a[0..i-1]`. Then `sum(l..r) = prefix[r+1] - prefix[l]`.

```
        static int[] buildPrefix(int[] a) {
            int[] p = new int[a.length + 1];
            for (int i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
            return p;
        }

        // sum of a[l..r] inclusive
        static int rangeSum(int[] p, int l, int r) {
            return p[r + 1] - p[l];
        }
```

Example: a = [2, 4, 6, 8], p = [0, 2, 6, 12, 20]. sum(1..3) = p[4] - p[1] = 20 - 2 = 18 (4 + 6 + 8).

Subarray sum equals K (prefix sum + HashMap)

Count subarrays with sum exactly k. If the prefix sum up to j is S and an earlier prefix was S - k, then the subarray between them sums to k.

```
static int subarraySum(int[] a, int k) {
    HashMap<Integer, Integer> seen = new HashMap<>();
    seen.put(0, 1);
    int sum = 0, count = 0;
    for (int x : a) {
        sum += x;
        count += seen.getOrDefault(sum - k, 0);
        seen.merge(sum, 1, Integer::sum);
    }
    return count;
}
```

Time O(n), space O(n). This works with negative numbers too.

2.4 Two Pointers

Use two indices that move toward each other (or in the same direction) to avoid a nested loop.

Two sum in a sorted array

```
static int[] twoSumSorted(int[] a, int target) {
    int l = 0, r = a.length - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == target) return new int[]{l, r};
        if (s < target) l++;
        else r--;
    }
    return new int[]{-1, -1};
}
```

Time O(n), space O(1). Why it works: if the sum is too small, only moving `l` right can increase it; if too large, only moving `r` left can decrease it.

Remove duplicates from a sorted array (in place)

```
static int removeDuplicates(int[] a) {
    if (a.length == 0) return 0;
    int w = 1;                          // write index
    for (int i = 1; i < a.length; i++) {
        if (a[i] != a[i - 1]) a[w++] = a[i];
    }
    return w;                           // new length
}
```

Move zeros to the end

```
static void moveZeros(int[] a) {
    int w = 0;
    for (int i = 0; i < a.length; i++) {
        if (a[i] != 0) { int t = a[w]; a[w] = a[i]; a[i] = t; w++; }
    }
}
```

Dutch national flag (sort 0s, 1s, 2s)

```
static void sortColors(int[] a) {
    int low = 0, mid = 0, high = a.length - 1;
```

```
        while (mid <= high) {
            if (a[mid] == 0) { swap(a, low++, mid++); }
            else if (a[mid] == 1) { mid++; }
            else { swap(a, mid, high--); }
        }
    }
    static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
```

One pass, O(n) time, O(1) space.

2.5 Sliding Window

Maintain a window `[l, r]` that slides over the array, updating the answer incrementally instead of recomputing.

Fixed window: maximum sum of k consecutive elements

```
    static int maxSumK(int[] a, int k) {
        int sum = 0;
        for (int i = 0; i < k; i++) sum += a[i];
        int best = sum;
        for (int i = k; i < a.length; i++) {
            sum += a[i] - a[i - k];        // add new element, drop old one
            best = Math.max(best, sum);
        }
        return best;
    }
```

Time O(n).

Variable window: smallest subarray with sum >= target (positive numbers)

```
    static int minSubarrayLen(int target, int[] a) {
        int l = 0, sum = 0, best = Integer.MAX_VALUE;
        for (int r = 0; r < a.length; r++) {
            sum += a[r];
            while (sum >= target) {
                best = Math.min(best, r - l + 1);
                sum -= a[l++];
            }
        }
        return best == Integer.MAX_VALUE ? 0 : best;
    }
```

Each element enters and leaves the window at most once, so time is O(n).

2.6 Kadane's Algorithm (Maximum Subarray)

Find the contiguous subarray with the largest sum.

Idea: at each position, either extend the previous subarray or start fresh.

```
    static int maxSubarray(int[] a) {
        int cur = a[0], best = a[0];
        for (int i = 1; i < a.length; i++) {
            cur = Math.max(a[i], cur + a[i]);
            best = Math.max(best, cur);
        }
        return best;
    }
```

Example: [-2, 1, -3, 4, -1, 2, 1, -5, 4] gives 6 (subarray [4, -1, 2, 1]). Time O(n), space O(1).

2.7 Matrix Problems

Transpose and rotate 90 degrees clockwise (n x n)

Rotate = transpose, then reverse each row.

```
static void rotate90(int[][] m) {
    int n = m.length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t;
        }
    for (int[] row : m) {
        for (int l = 0, r = n - 1; l < r; l++, r--) {
            int t = row[l]; row[l] = row[r]; row[r] = t;
        }
    }
}
```

Spiral traversal

```
static List<Integer> spiral(int[][] m) {
    List<Integer> res = new ArrayList<>();
    int top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) res.add(m[top][j]);
        top++;
        for (int i = top; i <= bottom; i++) res.add(m[i][right]);
        right--;
        if (top <= bottom) {
            for (int j = right; j >= left; j--) res.add(m[bottom][j]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) res.add(m[i][left]);
            left++;
        }
    }
    return res;
}
```

2.8 Strings in Java

`String` is **immutable**. Every "modification" creates a new object.

```
String s = "hello";
s.length();            // 5
s.charAt(1);           // 'e'
s.substring(1, 4);     // "ell" (end index exclusive)
s.indexOf('l');        // 2
s.contains("ell");     // true
s.toUpperCase();
s.equals("hello");     // compare content, never use == for this
s.compareTo("help");   // lexicographic comparison
s.toCharArray();       // char[]
String.valueOf(charArray);
s.split(" ");          // String[]
String.join(",", list);
```

StringBuilder (mutable)

```
StringBuilder sb = new StringBuilder();
sb.append("abc").append('d').append(12);
sb.insert(0, "X");        // O(n)
sb.reverse();
sb.setCharAt(0, 'Z');
sb.deleteCharAt(sb.length() - 1);
String result = sb.toString();
```

Character tricks

```
Character.isDigit(c);  Character.isLetter(c);  Character.isLetterOrDigit(c);
Character.toLowerCase(c);
int idx = c - 'a';             // 'a' -> 0, 'b' -> 1, ..., 'z' -> 25
char ch = (char) ('a' + 3);    // 'd'
```

2.9 Common String Problems

Check palindrome (ignoring case and non-alphanumerics)

```
static boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
        while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
        if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r)))
            return false;
        l++; r--;
    }
    return true;
}
```

Valid anagram (frequency array)

```
static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] freq = new int[26];
    for (int i = 0; i < a.length(); i++) {
        freq[a.charAt(i) - 'a']++;
        freq[b.charAt(i) - 'a']--;
    }
    for (int f : freq) if (f != 0) return false;
    return true;
}
```

Time O(n), space O(1) (26 slots).

First non-repeating character

```
static int firstUnique(String s) {
    int[] freq = new int[26];
    for (char c : s.toCharArray()) freq[c - 'a']++;
    for (int i = 0; i < s.length(); i++)
        if (freq[s.charAt(i) - 'a'] == 1) return i;
    return -1;
}
```

Longest substring without repeating characters (sliding window)

```
static int lengthOfLongestSubstring(String s) {
    HashMap<Character, Integer> last = new HashMap<>();
    int l = 0, best = 0;
    for (int r = 0; r < s.length(); r++) {
```

```
            char c = s.charAt(r);
            if (last.containsKey(c) && last.get(c) >= l) l = last.get(c) + 1;
            last.put(c, r);
            best = Math.max(best, r - l + 1);
        }
        return best;
    }
```

Time O(n).

Longest common prefix

```
    static String longestCommonPrefix(String[] strs) {
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (!strs[i].startsWith(prefix)) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }
```

Reverse words in a string

```
    static String reverseWords(String s) {
        String[] words = s.trim().split("\\s+");
        StringBuilder sb = new StringBuilder();
        for (int i = words.length - 1; i >= 0; i--) {
            sb.append(words[i]);
            if (i > 0) sb.append(' ');
        }
        return sb.toString();
    }
```

2.10 Pattern Searching (Naive and KMP)

Naive: O(n x m)

```
    static int indexOfNaive(String text, String pat) {
        int n = text.length(), m = pat.length();
        for (int i = 0; i + m <= n; i++) {
            int j = 0;
            while (j < m && text.charAt(i + j) == pat.charAt(j)) j++;
            if (j == m) return i;
        }
        return -1;
    }
```

KMP: O(n + m)

KMP precomputes the **LPS array** (longest proper prefix that is also a suffix) so that on a mismatch we never move backward in the text.

```
    static int[] buildLps(String p) {
        int[] lps = new int[p.length()];
        int len = 0;
        for (int i = 1; i < p.length(); ) {
            if (p.charAt(i) == p.charAt(len)) lps[i++] = ++len;
            else if (len > 0) len = lps[len - 1];
            else lps[i++] = 0;
        }
        return lps;
    }
    static int kmp(String text, String pat) {
```

```
            int[] lps = buildLps(pat);
            int i = 0, j = 0;
            while (i < text.length()) {
                if (text.charAt(i) == pat.charAt(j)) { i++; j++; }
                if (j == pat.length()) return i - j;
                if (i < text.length() && text.charAt(i) != pat.charAt(j)) {
                    if (j > 0) j = lps[j - 1];
                    else i++;
                }
            }
            return -1;
        }
```

2.11 Common Mistakes

1. Using `==` to compare strings. Use `.equals()`.
2. Concatenating strings in a loop. Use `StringBuilder`.
3. Forgetting that `substring(a, b)` excludes index `b`.
4. Off-by-one errors in loops and window boundaries.
5. Using `int` for sums that can overflow. Use `long`.
6. Modifying a list while iterating over it (`ConcurrentModificationException`).
7. Assuming the string contains only lowercase letters when it may not.

2.12 Chapter Summary

• Arrays give O(1) index access; inserts and deletes in the middle are O(n).
• **Prefix sums** answer range queries in O(1).
• **Two pointers** and **sliding window** turn many $O(n^2)$ solutions into O(n).
• **Kadane's algorithm** solves maximum subarray in O(n).
• Strings are immutable; use `StringBuilder` for building.
• Frequency arrays (size 26 or 256) are the fastest way to count characters.

2.13 Practice Problems

1. Find the missing number in an array containing n distinct numbers from 0 to n.
2. Find the maximum product subarray.
3. Merge two sorted arrays into one sorted array.
4. Given an array, return the product of all other elements for each index without using division.
5. Find the majority element (appears more than n/2 times) in O(n) time and O(1) space.
6. Check whether two strings are rotations of each other.
7. Find the longest palindromic substring.
8. Group anagrams from a list of words.

2.14 Solutions

1. Missing number. Expected sum minus actual sum.

```
static int missing(int[] a) {
    int n = a.length;
    int expected = n * (n + 1) / 2, actual = 0;
    for (int x : a) actual += x;
    return expected - actual;
}
```

Or XOR all indices and values. O(n) time, O(1) space.

2. Max product subarray. Track both max and min product ending here, since a negative can flip them.

```
static int maxProduct(int[] a) {
    int max = a[0], min = a[0], best = a[0];
    for (int i = 1; i < a.length; i++) {
        int x = a[i];
        if (x < 0) { int t = max; max = min; min = t; }
        max = Math.max(x, max * x);
        min = Math.min(x, min * x);
        best = Math.max(best, max);
    }
    return best;
}
```

3. Merge sorted arrays.

```
static int[] merge(int[] a, int[] b) {
    int[] r = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length) r[k++] = a[i] <= b[j] ? a[i++] : b[j++];
    while (i < a.length) r[k++] = a[i++];
    while (j < b.length) r[k++] = b[j++];
    return r;
}
```

4. Product except self. Left products pass, then right products pass.

```
static int[] productExceptSelf(int[] a) {
    int n = a.length;
    int[] res = new int[n];
    res[0] = 1;
    for (int i = 1; i < n; i++) res[i] = res[i - 1] * a[i - 1];
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        res[i] *= right;
        right *= a[i];
    }
    return res;
}
```

5. Majority element (Boyer-Moore voting).

```
static int majority(int[] a) {
    int cand = 0, count = 0;
    for (int x : a) {
        if (count == 0) cand = x;
        count += (x == cand) ? 1 : -1;
    }
    return cand;   // valid if a majority element is guaranteed
}
```

6. String rotation. b is a rotation of a exactly when lengths match and `(a + a).contains(b)`.

7. Longest palindromic substring (expand around center), O(n²).

```
static String longestPalindrome(String s) {
```

```
            int start = 0, end = 0;
            for (int i = 0; i < s.length(); i++) {
                int len = Math.max(expand(s, i, i), expand(s, i, i + 1));
                if (len > end - start + 1) {
                    start = i - (len - 1) / 2;
                    end = i + len / 2;
                }
            }
            return s.substring(start, end + 1);
        }
        static int expand(String s, int l, int r) {
            while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
            return r - l - 1;
        }
```

8. Group anagrams. Sort each word's characters to build a key.

```
        static List<List<String>> groupAnagrams(String[] words) {
            HashMap<String, List<String>> map = new HashMap<>();
            for (String w : words) {
                char[] c = w.toCharArray();
                Arrays.sort(c);
                map.computeIfAbsent(new String(c), k -> new ArrayList<>()).add(w);
            }
            return new ArrayList<>(map.values());
        }
```

Next: Chapter 3, Recursion and Backtracking

Chapter 3: Recursion and Backtracking

3.1 What Is Recursion?

Recursion is when a function **calls itself** to solve a smaller version of the same problem.

Every recursive function needs two things:

1. **Base case:** the simplest input, answered directly. Without it, recursion never stops (`StackOverflowError`).

2. **Recursive case:** reduce the problem toward the base case and combine results.

```
static int factorial(int n) {
    if (n <= 1) return 1;            // base case
    return n * factorial(n - 1);     // recursive case
}
```

How the call stack works

`factorial(3)` calls `factorial(2)`, which calls `factorial(1)`. Each call waits on the stack until the one above it returns:

```
factorial(3) -> 3 * factorial(2)
                    factorial(2) -> 2 * factorial(1)
                                        factorial(1) -> 1
```

Then results flow back: 1, then 2, then 6.

A 3-step method for writing recursion

1. **Define** exactly what the function returns for input `n`.

2. **Trust** that it works for smaller inputs.

3. **Combine** the smaller answer to get the answer for `n`, then add the base case.

3.2 Classic Recursion Examples

Sum of digits

```
static int digitSum(int n) {
    if (n == 0) return 0;
    return n % 10 + digitSum(n / 10);
}
```

Power (fast exponentiation) in O(log n)

```
static long power(long x, int n) {
    if (n == 0) return 1;
    long half = power(x, n / 2);
    if (n % 2 == 0) return half * half;
    return half * half * x;
}
```

Naive multiplication is O(n); halving the exponent gives O(log n).

Fibonacci: bad and good

```
// Bad: O(2^n)
static int fibSlow(int n) {
    if (n <= 1) return n;
    return fibSlow(n - 1) + fibSlow(n - 2);
}

// Good: O(n) using memoization
static long fib(int n, long[] memo) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
}
```

The slow version recomputes the same values repeatedly. **Memoization** stores results so each is computed once. This idea is the foundation of Dynamic Programming (Chapter 12).

Reverse a string recursively

```
static String reverse(String s) {
    if (s.length() <= 1) return s;
    return reverse(s.substring(1)) + s.charAt(0);
}
```

(Educational only; the iterative version is more efficient because of repeated substring copies.)

Check palindrome recursively

```
static boolean isPal(String s, int l, int r) {
    if (l >= r) return true;
    if (s.charAt(l) != s.charAt(r)) return false;
    return isPal(s, l + 1, r - 1);
}
```

Tower of Hanoi

Move n disks from `src` to `dest` using `aux`, never placing a larger disk on a smaller one.

```
static void hanoi(int n, char src, char dest, char aux) {
    if (n == 0) return;
    hanoi(n - 1, src, aux, dest);
    System.out.println("Move disk " + n + " from " + src + " to " + dest);
    hanoi(n - 1, aux, dest, src);
}
```

Moves needed: 2^n - 1, so time is O(2^n).

Greatest common divisor (Euclid)

```
static int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

Time O(log(min(a, b))).

3.3 Recursion Tree and Complexity

Draw the calls as a tree. Time = (number of nodes) x (work per node). Space = depth of the tree (stack).

Recurrence	Time	Example

T(n) = T(n-1) + O(1)	O(n)	factorial, sum
T(n) = T(n/2) + O(1)	O(log n)	binary search, fast power
T(n) = 2T(n-1) + O(1)	O(2^n)	naive Fibonacci-like, Hanoi
T(n) = 2T(n/2) + O(n)	O(n log n)	merge sort

Tail recursion note

Some languages optimize tail recursion; **Java does not**. Very deep recursion (around 10^4 to 10^5 frames, depending on frame size) can throw `StackOverflowError`. If depth can be large, convert to a loop or use an explicit stack.

3.4 Backtracking

Backtracking = **try a choice, explore, undo the choice (backtrack), try the next**. It systematically explores all possibilities while **pruning** branches that cannot lead to a valid answer.

The universal template

```
void backtrack(state, choices) {
    if (state is a complete solution) {
        record it;
        return;
    }
    for (choice : choices) {
        if (!isValid(choice)) continue;     // pruning
        make(choice);                       // choose
        backtrack(newState, remainingChoices);   // explore
        undo(choice);                       // un-choose
    }
}
```

The "choose, explore, un-choose" trio is the whole idea.

3.5 Subsets (Power Set)

Generate all 2^n subsets of an array of distinct integers.

```
static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), res);
    return res;
}

static void backtrack(int[] nums, int start, List<Integer> cur, List<List<Integer>> res) {
    res.add(new ArrayList<>(cur));            // every node is a valid subset
    for (int i = start; i < nums.length; i++) {
        cur.add(nums[i]);                     // choose
        backtrack(nums, i + 1, cur, res);     // explore
        cur.remove(cur.size() - 1);           // un-choose
    }
}
```

Important: `res.add(new ArrayList<>(cur))` copies the list. If you add `cur` itself, all stored results will change as `cur` changes.

Time O(n x 2^n), space O(n) for recursion (excluding output).

Subsets with duplicates

Sort first, then skip equal elements at the same level.

```
        static void backtrackDup(int[] nums, int start, List<Integer> cur, List<List<Integer>> res) {
            res.add(new ArrayList<>(cur));
            for (int i = start; i < nums.length; i++) {
                if (i > start && nums[i] == nums[i - 1]) continue;   // skip duplicates
                cur.add(nums[i]);
                backtrackDup(nums, i + 1, cur, res);
                cur.remove(cur.size() - 1);
            }
        }
```

(Call `Arrays.sort(nums)` before starting.)

3.6 Permutations

Generate all n! orderings of distinct numbers.

```
        static List<List<Integer>> permute(int[] nums) {
            List<List<Integer>> res = new ArrayList<>();
            permute(nums, new boolean[nums.length], new ArrayList<>(), res);
            return res;
        }

        static void permute(int[] nums, boolean[] used, List<Integer> cur, List<List<Integer>> res) {
            if (cur.size() == nums.length) {
                res.add(new ArrayList<>(cur));
                return;
            }
            for (int i = 0; i < nums.length; i++) {
                if (used[i]) continue;
                used[i] = true;
                cur.add(nums[i]);
                permute(nums, used, cur, res);
                cur.remove(cur.size() - 1);
                used[i] = false;
            }
        }
```

Time O(n x n!).

Swap-based permutations (no `used` array)

```
        static void permuteSwap(int[] a, int idx, List<List<Integer>> res) {
            if (idx == a.length) {
                List<Integer> p = new ArrayList<>();
                for (int x : a) p.add(x);
                res.add(p);
                return;
            }
            for (int i = idx; i < a.length; i++) {
                swap(a, idx, i);
                permuteSwap(a, idx + 1, res);
                swap(a, idx, i);          // backtrack
            }
        }
        static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
```

3.7 Combinations and Combination Sum

Combinations: choose k numbers from 1..n

```
        static void combine(int n, int k, int start, List<Integer> cur, List<List<Integer>> res) {
            if (cur.size() == k) { res.add(new ArrayList<>(cur)); return; }
            for (int i = start; i <= n; i++) {
                cur.add(i);
                combine(n, k, i + 1, cur, res);
                cur.remove(cur.size() - 1);
            }
        }
```

Combination Sum (numbers may be reused)

Find all combinations from `cand[]` that sum to `target`.

```
        static void combSum(int[] cand, int target, int start, List<Integer> cur, List<List<Integer>> res) {
            if (target == 0) { res.add(new ArrayList<>(cur)); return; }
            for (int i = start; i < cand.length; i++) {
                if (cand[i] > target) continue;       // prune (use break if cand is sorted)
                cur.add(cand[i]);
                combSum(cand, target - cand[i], i, cur, res);   // i, not i+1: reuse allowed
                cur.remove(cur.size() - 1);
            }
        }
```

3.8 Generate Valid Parentheses

Generate all well-formed strings with n pairs of parentheses.

Rule: add `(` if we still have some left; add `)` only if it won't exceed the number of `(` used.

```
        static void gen(int open, int close, int n, StringBuilder sb, List<String> res) {
            if (sb.length() == 2 * n) { res.add(sb.toString()); return; }
            if (open < n) {
                sb.append('(');
                gen(open + 1, close, n, sb, res);
                sb.deleteCharAt(sb.length() - 1);
            }
            if (close < open) {
                sb.append(')');
                gen(open, close + 1, n, sb, res);
                sb.deleteCharAt(sb.length() - 1);
            }
        }
```

The number of results is the n-th Catalan number.

3.9 N-Queens

Place n queens on an n x n board so that no two attack each other (same row, column, or diagonal).

Place one queen per row, and keep boolean arrays for used columns and the two diagonals.

• Column `c` is taken: `cols[c]`

• Diagonal (row - col) is constant along "\\" diagonals: `d1[row - col + n - 1]`

• Anti-diagonal (row + col) is constant along "/" diagonals: `d2[row + col]`

```
        static List<List<String>> solveNQueens(int n) {
```

```
        List<List<String>> res = new ArrayList<>();
        char[][] board = new char[n][n];
        for (char[] row : board) Arrays.fill(row, '.');
        place(0, n, board, new boolean[n], new boolean[2 * n - 1], new boolean[2 * n - 1], res);
        return res;
    }

    static void place(int row, int n, char[][] board, boolean[] cols,
                      boolean[] d1, boolean[] d2, List<List<String>> res) {
        if (row == n) {
            List<String> sol = new ArrayList<>();
            for (char[] r : board) sol.add(new String(r));
            res.add(sol);
            return;
        }
        for (int c = 0; c < n; c++) {
            if (cols[c] || d1[row - c + n - 1] || d2[row + c]) continue;
            board[row][c] = 'Q';
            cols[c] = d1[row - c + n - 1] = d2[row + c] = true;
            place(row + 1, n, board, cols, d1, d2, res);
            board[row][c] = '.';
            cols[c] = d1[row - c + n - 1] = d2[row + c] = false;
        }
    }
```

For n = 4 there are 2 solutions; for n = 8 there are 92.

3.10 Sudoku Solver

Fill empty cells (`'.'`) so every row, column and 3x3 box contains digits 1 to 9 once.

```
    static boolean solve(char[][] b) {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (b[r][c] != '.') continue;
                for (char d = '1'; d <= '9'; d++) {
                    if (isValid(b, r, c, d)) {
                        b[r][c] = d;
                        if (solve(b)) return true;
                        b[r][c] = '.';            // backtrack
                    }
                }
                return false;                     // no digit fits: dead end
            }
        }
        return true;                              // no empty cell left
    }

    static boolean isValid(char[][] b, int r, int c, char d) {
        for (int i = 0; i < 9; i++) {
            if (b[r][i] == d || b[i][c] == d) return false;
            int br = 3 * (r / 3) + i / 3, bc = 3 * (c / 3) + i % 3;
            if (b[br][bc] == d) return false;
        }
        return true;
    }
```

3.11 Grid Backtracking: Word Search

Does a word exist in a grid by moving to adjacent cells (no cell reused)?

```
    static boolean exist(char[][] g, String word) {
        for (int i = 0; i < g.length; i++)
            for (int j = 0; j < g[0].length; j++)
                if (dfs(g, word, i, j, 0)) return true;
        return false;
```

```
        }

        static boolean dfs(char[][] g, String w, int i, int j, int k) {
            if (k == w.length()) return true;
            if (i < 0 || j < 0 || i >= g.length || j >= g[0].length || g[i][j] != w.charAt(k))
                return false;
            char saved = g[i][j];
            g[i][j] = '#';                          // mark visited
            boolean found = dfs(g, w, i + 1, j, k + 1) || dfs(g, w, i - 1, j, k + 1)
                         || dfs(g, w, i, j + 1, k + 1) || dfs(g, w, i, j - 1, k + 1);
            g[i][j] = saved;                        // un-mark (backtrack)
            return found;
        }
```

3.12 Recursion vs Iteration

	Recursion	Iteration
Code	Often shorter, mirrors the problem	Can be longer
Memory	Uses call stack	Usually O(1) extra
Risk	StackOverflowError on deep inputs	None
Best for	Trees, graphs, divide and conquer, backtracking	Simple loops, performance-critical paths

---

3.13 Common Mistakes

1. Missing or wrong **base case**, causing infinite recursion.
2. Not making the problem **smaller** on each call.
3. Forgetting to **undo** the choice in backtracking.
4. Adding `cur` directly to results instead of a **copy**.
5. Recomputing overlapping subproblems (use memoization).
6. Not **pruning**, making a solution far slower than necessary.
7. Handling duplicates incorrectly (sort, then skip equal values at the same level).

3.14 Chapter Summary

• Recursion = base case + smaller subproblem + combine.

• Stack space is proportional to recursion depth.

• Memoization removes repeated work (leads to DP).

• Backtracking template: **choose, explore, un-choose**.

• Subsets, permutations, combinations, N-Queens, Sudoku and word search all follow the same template with different validity rules.

• Pruning is what makes backtracking practical.

3.15 Practice Problems

1. Print numbers from n down to 1 and then from 1 to n using recursion.
2. Count the number of ways to climb n stairs taking 1 or 2 steps at a time.
3. Generate all binary strings of length n with no two consecutive 1s.
4. Generate all letter combinations of a phone-keypad digit string (e.g., "23").
5. Partition a string into all possible palindromic substrings.
6. Find all subsets of an array that sum to a target.
7. Solve the "rat in a maze" problem: find all paths from (0,0) to (n-1,n-1) moving D, L, R, U through cells equal to 1.
8. Generate all permutations of a string containing duplicate characters without repeating results.

3.16 Solutions

1.

```
static void print(int n) {
    if (n == 0) return;
    System.out.print(n + " ");
    print(n - 1);
}
// then 1..n: call print2(n) where print2 recurses first and prints after.
static void print2(int n) {
    if (n == 0) return;
    print2(n - 1);
    System.out.print(n + " ");
}
```

2. Climbing stairs. ways(n) = ways(n-1) + ways(n-2).

```
static int climb(int n, int[] memo) {
    if (n <= 1) return 1;
    if (memo[n] != 0) return memo[n];
    return memo[n] = climb(n - 1, memo) + climb(n - 2, memo);
}
```

3. No consecutive 1s.

```
static void gen(int n, StringBuilder sb, List<String> res) {
    if (sb.length() == n) { res.add(sb.toString()); return; }
    sb.append('0'); gen(n, sb, res); sb.deleteCharAt(sb.length() - 1);
    if (sb.length() == 0 || sb.charAt(sb.length() - 1) != '1') {
        sb.append('1'); gen(n, sb, res); sb.deleteCharAt(sb.length() - 1);
    }
}
```

4. Phone keypad.

```
static final String[] MAP = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
static void letters(String digits, int i, StringBuilder sb, List<String> res) {
    if (digits.isEmpty()) return;
    if (i == digits.length()) { res.add(sb.toString()); return; }
    for (char c : MAP[digits.charAt(i) - '0'].toCharArray()) {
        sb.append(c);
        letters(digits, i + 1, sb, res);
        sb.deleteCharAt(sb.length() - 1);
    }
}
```

5. Palindrome partitioning.

```
static void partition(String s, int start, List<String> cur, List<List<String>> res) {
    if (start == s.length()) { res.add(new ArrayList<>(cur)); return; }
```

```
        for (int end = start; end < s.length(); end++) {
            if (isPal(s, start, end)) {
                cur.add(s.substring(start, end + 1));
                partition(s, end + 1, cur, res);
                cur.remove(cur.size() - 1);
            }
        }
    }
    static boolean isPal(String s, int l, int r) {
        while (l < r) if (s.charAt(l++) != s.charAt(r--)) return false;
        return true;
    }
```

6. Subset sum. Use the subsets template; at each node check whether the running sum equals the target (or pass `target - nums[i]` down and record when it hits 0).

7. Rat in a maze.

```
    static void rat(int[][] m, int r, int c, boolean[][] vis, StringBuilder path, List<String> res) {
        int n = m.length;
        if (r < 0 || c < 0 || r >= n || c >= n || m[r][c] == 0 || vis[r][c]) return;
        if (r == n - 1 && c == n - 1) { res.add(path.toString()); return; }
        vis[r][c] = true;
        String dirs = "DLRU"; int[] dr = {1, 0, 0, -1}, dc = {0, -1, 1, 0};
        for (int k = 0; k < 4; k++) {
            path.append(dirs.charAt(k));
            rat(m, r + dr[k], c + dc[k], vis, path, res);
            path.deleteCharAt(path.length() - 1);
        }
        vis[r][c] = false;
    }
```

8. Unique permutations with duplicates. Sort the characters; in the loop, skip index `i` if `used[i]`, or if `i > 0 && chars[i] == chars[i-1] && !used[i-1]`.

Next: Chapter 4, Searching and Sorting

Chapter 4: Searching and Sorting

4.1 Searching

Linear search: O(n)

Works on any array, sorted or not.

```
static int linearSearch(int[] a, int key) {
    for (int i = 0; i < a.length; i++)
        if (a[i] == key) return i;
    return -1;
}
```

Binary search: O(log n)

Requires a **sorted** array. Compare with the middle element and discard half each time.

```
static int binarySearch(int[] a, int key) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;      // avoids overflow of (lo + hi)
        if (a[mid] == key) return mid;
        if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Why `lo + (hi - lo) / 2`? If `lo` and `hi` are both near 2^31, `lo + hi` overflows `int`.

Recursive version:

```
static int bs(int[] a, int lo, int hi, int key) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == key) return mid;
    return a[mid] < key ? bs(a, mid + 1, hi, key) : bs(a, lo, mid - 1, key);
}
```

4.2 Binary Search Variants

First occurrence (lower bound)

```
static int firstOccurrence(int[] a, int key) {
    int lo = 0, hi = a.length - 1, ans = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == key) { ans = mid; hi = mid - 1; }   // keep looking left
        else if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return ans;
}
```

Last occurrence

Same, but on a match do `ans = mid; lo = mid + 1;`.

Count of a key = `last - first + 1`.

Lower bound / insertion position

Index of the first element >= key (also the position where key should be inserted).

```
static int lowerBound(int[] a, int key) {
    int lo = 0, hi = a.length;           // note: hi = length
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < key) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

Search in a rotated sorted array

One half of the array around `mid` is always sorted. Check which half, then decide where the key can be.

```
static int searchRotated(int[] a, int key) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == key) return mid;
        if (a[lo] <= a[mid]) {                      // left half sorted
            if (a[lo] <= key && key < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {                                    // right half sorted
            if (a[mid] < key && key <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Find peak element

```
static int findPeak(int[] a) {
    int lo = 0, hi = a.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < a[mid + 1]) lo = mid + 1;   // a peak lies to the right
        else hi = mid;
    }
    return lo;
}
```

4.3 Binary Search on the Answer

You can binary search not just on arrays but on a **range of possible answers**, whenever the condition is **monotonic**: if answer x works, then every larger (or smaller) value also works.

Template for "find the minimum x such that `feasible(x)` is true":

```
int lo = minPossible, hi = maxPossible;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (feasible(mid)) hi = mid;
    else lo = mid + 1;
}
return lo;
```

Example: integer square root

```
static int mySqrt(int x) {
    long lo = 0, hi = x;
    while (lo < hi) {
        long mid = (lo + hi + 1) / 2;        // upper mid to avoid infinite loop
        if (mid * mid <= x) lo = mid;
        else hi = mid - 1;
    }
    return (int) lo;
}
```

Example: Koko eating bananas (minimum speed)

Piles of bananas; eat at speed k bananas/hour (one pile per hour max). Find the smallest k so all piles are finished within h hours.

```
static int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = 0;
    for (int p : piles) hi = Math.max(hi, p);
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        long hours = 0;
        for (int p : piles) hours += (p + mid - 1) / mid;   // ceil(p / mid)
        if (hours <= h) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

Time O(n log(max)).

Example: split array into m parts minimizing the largest sum (book allocation)

Feasibility: can we split so each part has sum <= limit using at most m parts? Binary search `limit` between `max(a)` and `sum(a)`.

```
static int splitArray(int[] a, int m) {
    int lo = 0, hi = 0;
    for (int x : a) { lo = Math.max(lo, x); hi += x; }
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (parts(a, mid) <= m) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
static int parts(int[] a, int limit) {
    int count = 1, sum = 0;
    for (int x : a) {
        if (sum + x > limit) { count++; sum = 0; }
        sum += x;
    }
    return count;
}
```

4.4 Sorting Overview

Algorithm	Best	Average	Worst	Space	Stable?
Bubble	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes

Quick	O(n log n)	O(n log n)	O(n²)	O(log n)	No
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting	O(n + k)	O(n + k)	O(n + k)	O(k)	Yes

A sort is **stable** if equal elements keep their original relative order. A **comparison-based** sort cannot beat O(n log n) in the worst case.

4.5 Simple Sorts (O(n²))

Bubble sort

Repeatedly swap adjacent out-of-order elements. After pass `i`, the `i` largest elements are in place.

```
static void bubbleSort(int[] a) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = true;
            }
        }
        if (!swapped) break;           // already sorted: best case O(n)
    }
}
```

Selection sort

Select the minimum of the unsorted part and place it at the front.

```
static void selectionSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        int min = i;
        for (int j = i + 1; j < a.length; j++)
            if (a[j] < a[min]) min = j;
        int t = a[i]; a[i] = a[min]; a[min] = t;
    }
}
```

Makes at most n - 1 swaps, which is useful when writes are expensive.

Insertion sort

Like sorting playing cards in your hand: insert each element into its place in the sorted prefix.

```
static void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

Excellent for small or nearly sorted arrays. Real libraries use it for small chunks.

4.6 Merge Sort

Divide and conquer: split in half, sort each half recursively, **merge** the two sorted halves.

```
static void mergeSort(int[] a, int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(a, l, mid);
    mergeSort(a, mid + 1, r);
    merge(a, l, mid, r);
}

static void merge(int[] a, int l, int mid, int r) {
    int[] tmp = new int[r - l + 1];
    int i = l, j = mid + 1, k = 0;
    while (i <= mid && j <= r)
        tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];     // <= keeps it stable
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= r)   tmp[k++] = a[j++];
    System.arraycopy(tmp, 0, a, l, tmp.length);
}
```

Time: T(n) = 2T(n/2) + O(n) = **O(n log n)** in all cases. Space O(n).

Counting inversions with merge sort

An inversion is a pair `i < j` with `a[i] > a[j]`. While merging, if `a[j]` is taken before the remaining left elements, it forms inversions with all of them.

```
static long inversions = 0;
static void merge(int[] a, int l, int mid, int r) {
    int[] tmp = new int[r - l + 1];
    int i = l, j = mid + 1, k = 0;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else { tmp[k++] = a[j++]; inversions += (mid - i + 1); }
    }
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= r)   tmp[k++] = a[j++];
    System.arraycopy(tmp, 0, a, l, tmp.length);
}
```

4.7 Quick Sort

Pick a **pivot**, **partition** so smaller elements are left of it and larger are right, then sort each side recursively.

```
static void quickSort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);
    quickSort(a, p + 1, hi);
}

// Lomuto partition with a random pivot
static int partition(int[] a, int lo, int hi) {
    int r = lo + (int) (Math.random() * (hi - lo + 1));
    swap(a, r, hi);                       // move random pivot to the end
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) swap(a, i++, j);
    }
    swap(a, i, hi);
    return i;
}
static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
```

• **Average:** O(n log n). **Worst:** O(n²), when the pivot is always the smallest or largest (for example, a sorted array with a fixed last-element pivot).

• A **random pivot** makes the worst case extremely unlikely.

• Space: O(log n) average recursion depth.

• In practice quick sort is often faster than merge sort because of good cache behavior.

Quickselect: k-th smallest in average O(n)

```
    static int kthSmallest(int[] a, int k) {          // k is 1-based
        int lo = 0, hi = a.length - 1;
        while (true) {
            int p = partition(a, lo, hi);
            if (p == k - 1) return a[p];
            if (p < k - 1) lo = p + 1;
            else hi = p - 1;
        }
    }
```

4.8 Counting Sort (non-comparison)

When values lie in a small range `[0, k]`, count occurrences and rebuild the array.

```
    static void countingSort(int[] a, int k) {
        int[] count = new int[k + 1];
        for (int x : a) count[x]++;
        int idx = 0;
        for (int v = 0; v <= k; v++)
            while (count[v]-- > 0) a[idx++] = v;
    }
```

Time O(n + k), space O(k). Great when k is about the same size as n. Radix sort extends this idea digit by digit.

4.9 Sorting in Java

```
    int[] a = {5, 2, 9, 1};
    Arrays.sort(a);                          // primitives: dual-pivot quicksort, O(n log n)

    Integer[] b = {5, 2, 9, 1};
    Arrays.sort(b, Collections.reverseOrder());   // objects: stable TimSort

    List<Integer> list = new ArrayList<>(List.of(5, 2, 9, 1));
    Collections.sort(list);
    list.sort((x, y) -> y - x);              // descending (careful: x - y can overflow)
```

Sorting with a Comparator

```
    String[] words = {"banana", "kiwi", "apple", "fig"};
    Arrays.sort(words, (x, y) -> x.length() - y.length());           // by length
    Arrays.sort(words, Comparator.comparing(String::length)
                               .thenComparing(Comparator.naturalOrder()));

    int[][] intervals = {{5, 8}, {1, 3}, {2, 6}};
    Arrays.sort(intervals, (x, y) -> Integer.compare(x[0], y[0]));   // by start
```

Tip: Use `Integer.compare(x, y)` instead of `x - y`; subtraction can overflow for extreme values.

Sorting `int[]` in descending order: `Arrays.sort` with a comparator only works on object arrays. Either sort ascending and reverse, or use `Integer[]`.

4.10 Classic Problems Using Sorting

Merge overlapping intervals

```
static int[][] mergeIntervals(int[][] iv) {
    Arrays.sort(iv, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> res = new ArrayList<>();
    int[] cur = iv[0];
    for (int i = 1; i < iv.length; i++) {
        if (iv[i][0] <= cur[1]) cur[1] = Math.max(cur[1], iv[i][1]);
        else { res.add(cur); cur = iv[i]; }
    }
    res.add(cur);
    return res.toArray(new int[0][]);
}
```

Three-sum (sort + two pointers), O(n²)

```
static List<List<Integer>> threeSum(int[] a) {
    Arrays.sort(a);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < a.length - 2; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;
        int l = i + 1, r = a.length - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (s == 0) {
                res.add(List.of(a[i], a[l], a[r]));
                l++; r--;
                while (l < r && a[l] == a[l - 1]) l++;
                while (l < r && a[r] == a[r + 1]) r--;
            } else if (s < 0) l++;
            else r--;
        }
    }
    return res;
}
```

Largest number from array (custom comparator)

Given numbers like [3, 30, 34, 5, 9], arrange them to form the largest number ("9534330").

```
static String largestNumber(int[] nums) {
    String[] s = new String[nums.length];
    for (int i = 0; i < nums.length; i++) s[i] = String.valueOf(nums[i]);
    Arrays.sort(s, (a, b) -> (b + a).compareTo(a + b));
    if (s[0].equals("0")) return "0";
    return String.join("", s);
}
```

4.11 Choosing a Sorting Algorithm

• Small or nearly sorted data: **insertion sort**.

• General purpose, stability needed: **merge sort** (or Java's object sort).

• General purpose, memory tight: **quick sort** or **heap sort**.

• Small integer range: **counting sort**.

• Need only the top-k or k-th element: **heap** or **quickselect**, no full sort required.

4.12 Common Mistakes

1. Using `(lo + hi) / 2` instead of `lo + (hi - lo) / 2`.
2. Infinite loops in binary search from wrong `lo`/`hi` updates (`mid` vs `mid + 1`).
3. Applying binary search to a non-monotonic condition.
4. Using `x - y` in comparators.
5. Expecting `Arrays.sort(int[], comparator)` to compile.
6. Forgetting that merge sort needs O(n) extra space.
7. Using a fixed first or last pivot in quick sort on sorted data.

4.13 Chapter Summary

• Binary search halves the search space: O(log n), requires sorted or monotonic data.
• "Binary search on the answer" solves many optimization problems.
• Bubble, selection and insertion sort are $O(n^2)$; merge, quick and heap are O(n log n).
• Merge sort is stable and always O(n log n); quick sort is fastest in practice with a random pivot.
• Counting sort beats O(n log n) for small value ranges.
• Many problems become easy after sorting (intervals, three-sum, closest pairs).

4.14 Practice Problems

1. Find the square root of a number up to 2 decimal places using binary search.
2. Find the minimum element in a rotated sorted array.
3. Find the number of times a sorted array is rotated.
4. Find the median of two sorted arrays in O(log(min(m, n))).
5. Sort an array of 0s, 1s and 2s in one pass (revisit Chapter 2).
6. Find the k-th largest element in an array.
7. Find the minimum number of platforms needed for a railway station given arrival and departure times.
8. Sort an array by the frequency of elements (higher frequency first; ties by smaller value).

4.15 Solutions

2. Minimum in rotated sorted array.

```
        static int findMin(int[] a) {
            int lo = 0, hi = a.length - 1;
```

```
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] > a[hi]) lo = mid + 1;     // min is in the right half
            else hi = mid;
        }
        return a[lo];
    }
```

3. The number of rotations equals the index of the minimum element found above.

4. Median of two sorted arrays (partition method).

```
    static double median(int[] A, int[] B) {
        if (A.length > B.length) return median(B, A);
        int m = A.length, n = B.length, lo = 0, hi = m;
        while (lo <= hi) {
            int i = (lo + hi) / 2, j = (m + n + 1) / 2 - i;
            int aL = i == 0 ? Integer.MIN_VALUE : A[i - 1];
            int aR = i == m ? Integer.MAX_VALUE : A[i];
            int bL = j == 0 ? Integer.MIN_VALUE : B[j - 1];
            int bR = j == n ? Integer.MAX_VALUE : B[j];
            if (aL <= bR && bL <= aR) {
                if ((m + n) % 2 == 1) return Math.max(aL, bL);
                return (Math.max(aL, bL) + Math.min(aR, bR)) / 2.0;
            } else if (aL > bR) hi = i - 1;
            else lo = i + 1;
        }
        throw new IllegalArgumentException();
    }
```

6. K-th largest. Use quickselect for the (n - k + 1)-th smallest, average O(n). Or a min-heap of size k, O(n log k) (see Chapter 9).

7. Minimum platforms. Sort arrivals and departures separately; sweep with two pointers.

```
    static int platforms(int[] arr, int[] dep) {
        Arrays.sort(arr); Arrays.sort(dep);
        int need = 0, best = 0, i = 0, j = 0;
        while (i < arr.length) {
            if (arr[i] <= dep[j]) { need++; i++; best = Math.max(best, need); }
            else { need--; j++; }
        }
        return best;
    }
```

8. Sort by frequency.

```
    static Integer[] sortByFreq(int[] a) {
        HashMap<Integer, Integer> f = new HashMap<>();
        for (int x : a) f.merge(x, 1, Integer::sum);
        Integer[] b = Arrays.stream(a).boxed().toArray(Integer[]::new);
        Arrays.sort(b, (x, y) -> f.get(x).equals(f.get(y)) ? Integer.compare(x, y)
                                                           : Integer.compare(f.get(y), f.get(x)));
        return b;
    }
```

Next: Chapter 5, Linked List